

Zihinsel Model: Yabancı Sistemi Anlama Yordamı

Ömer Faruk Yavuz

September 2026

Özet

Deneyimli bir mühendisin bir konuda yıllar önce öğrendiği kuralın tersine geçmesi, çoğunlukla bilgi eklenmesiyle değil, kuralın hangi kısıtlar altında doğru olduğunun görülmesiyle olur. Başlangıçta öğretilen kurallar bağlamdan arındırılmış, geniş çerçevede güvenli kurallardır; sahada bağlam, maliyet, zaman ve bakım kısıtları devreye girdiğinde bunların yerini ödünleşim (*trade-off*) alır. Bu geçiş sırasında yaşanan yön kaybı hissi, öğrenme literatüründe tanımlanmış bir olgudur, ancak hissin varlığı yeni kanaatin doğruluğunu göstermez: aynı his, doğru bir revizyonda da hatalı bir aşırı genellemede de aynı biçimde ortaya çıkar. Yabancı bir sistemi anlamamanın işleyen yöntemi, sistemi değiştirmeden önce zihinsel bir model kurmaktır; bu iş program anlama (*program comprehension*) adı altında incelenmiş, yerleşik bir araştırma alanına sahiptir. Zihinsel modelin kurulması yalnızca kafada yürütmeyele yapılmaz; sürüm geçmişi, testler ve çalıştırma, kafada yürütmeden daha güvenilir kanıt üretir. Kural-bağlam geçişinin en kırılgan yanı, terk edilen kuralın gördüğü işlevlerin yerine bir şey konmamasıdır. “Kural yoktur, bağlam vardır” cümlesi de bir ezberdir ve her alanda geçerli değildir; veri kaybı ve güvenlik gibi sert kısıtların bulunduğu yerlerde denge arayışı doğrudan hatadır.

Giriş

Bir mesleğin ilk yıllarında öğretilen kurallar normatiftir: “şu yapılmalı”, “bu yapılmamalı” biçiminde, koşul belirtmeden verilir. Bu kaçınılmazdır, çünkü koşulları değerlendirebilmek için önce koşulların ne olduğunu görmüş olmak gerekir. Yıllar sonra aynı kişi, aynı kuralın bazı bağlamlarda zarar verdiğini gözlemlediğinde iki şey aynı anda olur: eski kural geçersizleşir ve yerine henüz açıkça formüle edilmemiş bir şey geçer. Bu ara dönemde kişi hem eski gerekçelerini kullanamaz hem de yeni konumunu savunacak dili henüz kurmamıştır.

Bu notta ele alınan üç ayrı problem şunlardır. Birincisi, bu geçişin ne olduğu ve hangi kavramlarla adlandırıldığı. İkincisi, yabancı ya da kötü yazılmış bir sistemi anlamamanın yöntemi; bu, geçişin ham maddesini üreten pratiktir, çünkü kuralın nerede çalışmadığı ancak gerçek sistemler okunarak görülür. Üçüncüsü, geçişin doğrulanması problemi: kişinin kendi kanaat değişiminin bir derinleşme mi yoksa bir aşırı genelleme mi olduğunu nasıl ayırt edeceği.

Merkezî terimler şunlardır. **Zihinsel model** (*mental model*), bir kişinin bir sistemin nasıl çalıştığına dair kafasında taşıdığı, sistemin bileşenlerini ve aralarındaki ilişkileri temsil eden içsel yapıdır (Craik, 1943; Johnson-Laird, 1983). **Ödünleşim**, iki istenen özelliğin aynı anda en üst düzeyde sağlanamayacağını kabul edip aralarında bilinçli bir denge seçmektir. **Program anlama**, yazılım mühendisliğinde bir programcının kendi yazmadığı bir kodu nasıl kavradığını inceleyen araştırma alanıdır.

Notun kapsamı dışında kalan iki şey vardır: kanaat değişiminin duygusal yönetimi ve belirli bir mühendislik pratiğinin genel geçer bir savunusu. Not, bir pratiğin doğruluğunu değil, bir pratik hakkındaki kanaatin nasıl sınanacağını ele alır.

Zihinsel Model ve Program Anlama

Kavramın kökeni

Zihinsel model kavramı psikolog Kenneth Craik tarafından ortaya atılmıştır. Craik (1943), organizmanın dış dünyanın küçük ölçekli bir modelini kafasında taşıdığını ve olası eylemlerin sonuçlarını bu model üzerinde deneyerek eylemden önce sınadığını öne sürmüştür. Kavram daha sonra Johnson-Laird (1983) tarafından akıl yürütme psikolojisinde, Norman (1983) tarafından da insan-makine etkileşiminde geliştirilmiştir.

Mühendislik açısından önemli olan kısım, modelin eksiksiz olmasının beklenmemesidir. Norman, kullanıcıların zihinsel modellerinin tipik olarak eksik, kararsız ve batıl inanç içerir olduğunu, buna rağmen işlevsel olduğunu belirtmiştir. Bir modelin değeri doğruluğundan çok, doğru tahmin üretip üretmediğindedir.

Programlamanın teori kurma olarak görülmesi

Peter Naur (1985), programlamanın asıl ürününün kaynak kod değil, programcıların kafasında oluşan teori olduğunu savunmuştur. Naur'a göre kod ve dokümantasyon bu teorinin kısmi ve kayıplı bir yansımasıdır; teoriyi taşıyan ekip dağıldığında, kod elde kalsa bile program pratik olarak ölmüş olur, çünkü kodun neden öyle olduğunu açıklayacak yapı kaybolmuştur.

Bu görüş, yabancı bir kodu okuma probleminin neden zor olduğunu açıklar. Okuyucu yalnızca metni çözmez; metni üretmiş olan ve artık mevcut olmayan teoriyi yeniden kurmaya çalışır. Kodun “çirkin” görünmesi çoğu zaman kötü yazılmış olmasından değil, arkasındaki teorinin erişilemez olmasından kaynaklanır.

Anlama modelleri

Program anlama üzerine deneysel çalışmalar 1980'lerde yoğunlaşmıştır. Brooks (1983) anlamanın yukarıdan aşağı (*top-down*) bir hipotez sınama süreci olduğunu; okuyucunun kodun ne yaptığına dair bir üst hipotez kurup bunu alt hipotezlere böldüğünü ve kodda bu hipotezleri doğrulayacak işaretler (*beacons*) aradığını öne sürmüştür. Letovsky (1987), programcıların kod okurken ürettiği soruları ve varsayımları incelemiş, anlama sürecinin düzenli bir sıra izlemediğini, fırsatçı biçimde ilerlediğini göstermiştir. Von Mayrhauser ve Vans (1995) bu yaklaşımları birleştiren bir model önermiş; deneyimli programcıların tanıdık kodda yukarıdan aşağı, tanımadık kodda aşağıdan yukarı çalıştığını, ikisi arasında sürekli geçiş yaptığını belirtmişlerdir.

Buradan çıkan pratik sonuç, anlama işinin baştan sona doğrusal bir okuma olmadığıdır. Kodu satır satır takip etmek deneyimsiz okuyucunun yöntemidir; deneyimli okuyucu bir hipotez kurar, onu sımayacak birkaç noktaya bakar ve hipotezi düzeltir.

Occam'ın Usturası

Occam'ın usturası (*Occam's razor*), aynı olguyu açıklayan birden fazla açıklama arasından daha az varsayıma dayananı tercih etmeyi öngören bir düşünme kuralıdır. Adını on dördüncü yüzyıl filozofu William of Ockham'dan alır; "ustura" sözcüğü gereksiz varsayımların kesilip atılmasını ifade eder.

Kural, basit açıklamanın doğru olduğunu garanti etmez. Basit açıklamanın üstünlüğü, daha az noktadan kırılması ve daha kolay sınanabilmesidir. Karmaşık bir açıklama her eklenen varsayımla birlikte yeni bir yanlış olasılığı taşır.

Mühendislikteki karşılıkları:

- Bir hata gözlemlendiğinde "derleyicide hata var" ile "kodda yazım hatası var" açıklamaları arasında önce ikincisi sımanır.
- Bir ihtiyaç hem üç servis, bir kuyruk ve bir önbellekle hem de tek bir fonksiyonla karşılanabiliyorsa, tek fonksiyon tercih edilir.
- Bir veri tutarsızlığı için "ağ paketi bozuldu" yerine "iki bileşen farklı zaman dilimi kullanıyor" açıklaması hem daha basit hem daha olasıdır.

Emniyet Payı

Emniyet payı (*safety margin*), bir sistemin dayanabileceği en uç sınıra göre değil, bu sınırın belirli bir oranda gerisine göre tasarlanmasıdır. Sınır ile tasarım yükü arasındaki fark, hesaplanmamış sapmaları karşılamak için ayrılan boşluktur.

- Yüz ton taşıyabilen bir köprüye "en fazla elli ton" sınırı konur; aradaki elli ton emniyet payıdır.
- Saniyede bin istek kaldırabilen bir sunucu normal yükte altı yüz istekte tutulur; ani artış geldiğinde sistem çökmez.
- Yüz gigabaytlık bir disk için alarm eşiği seksen gigabayta kurulur; disk dolana kadar beklenmez.

Emniyet payının gerekçesi, hesapların her zaman bir miktar hatalı olmasıdır. Malzeme beklenenden zayıf çıkar, kullanıcı sayısı beklenenden fazla olur, kod beklenenden yavaş çalışır. Pay bırakılmadığında bu küçük sapmaların her biri sistemi düşürebilecek büyüklüğe ulaşır.

Büyüklik Sırası Tahmini

Büyüklik sırası tahmini (*order-of-magnitude estimation*), kesin değer bilinmeden bir niceliğin on mu, bin mi, yoksa bir milyon mu olduğunu belirleyebilme becerisidir. Aranacak şey ondalık hane değil, sıfır sayısıdır. Bu yaklaşım fizikçi Enrico Fermi'nin kaba hesap yöntemiyle ilişkilendirilir ve Fermi tahmini olarak da anılır.

- Bir tablonun kaplayacağı alan: satır başına yaklaşık 200 byte, on milyon satır; kabaca 2 GB. Kesin değer bu olmayabilir, ancak 20 MB ya da 200 GB olmadığı bellidir.
- Bir döngünün süresi: adım başına 1 ms, yüz bin adım; yaklaşık 100 saniye. Bu süre kullanıcı arayüzünde bekletilemez.
- Bir API'ye gelecek istek sayısı: on bin kullanıcı, kişi başı günde elli istek; günde 500.000, saniyede kabaca altı istek. Tek bir küçük sunucu yeterlidir.

Tahminin iki işlevi vardır. Birincisi, ayrıntılı hesaba girmeden doğru ölçeğin seçilmesini sağlar. İkincisi, elde edilen bir sonucun makul olup olmadığını denetler: tahmin üç saniye iken sorgu üç saat sürüyorsa bir yerde hata vardır.

Emniyet payı ve büyüklük sırası tahmini birlikte çalışır: önce kaba tahmin yapılır, ardından bu tahminin üzerine pay konur.

Sonuç ile Süreci Ayırmak

Bir kararın niteliği, ürettiği sonuca göre değil, verilmiş biçimine göre değerlendirilir. Sonuç iki bileşenden oluşur: karar ve şans. Şans karar vericinin denetiminde olmadığından, yalnızca sonuca bakarak değerlendirme yapmak yanlış ders çıkarılmasına yol açar. Bu hatayı Duke (2018) *resulting* terimiyle adlandırmıştır.

Dört durum ayırt edilir:

- **İyi süreç, iyi sonuç.** Testler yazılmış, kod ilk denemede çalışmıştır. Beklenen durumdur.
- **İyi süreç, kötü sonuç.** Testler yazılmış, yedek alınmış, planlı geçiş yapılmıştır; yine de o gün veri merkezinde elektrik kesilmiştir. Karar doğru, şans kötüdür. Süreç değiştirilmez.
- **Kötü süreç, iyi sonuç.** Cuma akşamı test edilmeden canlıya çıkılmış, hiçbir şey bozulmamıştır. Bu, kararın doğru olduğunu göstermez; aynı davranış tekrarlandığında bir gün başarısız olur.
- **Kötü süreç, kötü sonuç.** Test edilmeden çıkılmış, sistem çökmüştür. Beklenen durumdur.

Tuzak üçüncü durumdadır. Şanslı sonuç kötü alışkanlığı ödüllendirir; birkaç sorunsuz tekrar “gerek yok” yargısını üretir, ardından büyük bir kaza gelir.

Bir karar tek seferlik değil, yüz kez tekrarlanacak gibi düşünülmelidir. Yüz tekrarın ortalamasında iyi sonuç veren süreç iyi süreçtir; tek bir sonuç bunu göstermez. Pratikteki ölçüt şudur: o anda eldeki bilgiyle karar makul muydu? Cevap evetse sonuç kötü çıksa bile süreç iyidir; hayırsa sonuç iyi çıksa bile süreç kötüdür.

Yerel Optimum Tuzağı

Bir parçayı en iyi hâline getirmek, bütünü en iyi hâline getirmek anlamına gelmez; bazı durumlarda bütünü kötüleştirir. Bu ilke, Goldratt'ın kısıtlar teorisinde (*Theory of Constraints*) merkezi yer tutar: bir sistemin çıktısını yalnızca darboğazdaki iyileştirme artırır, darboğaz dışındaki iyileştirmeler bütüne katkı sağlamaz (Goldratt ve Cox, 1984).

- Veritabanı ekibi sorguları hızlandırmak için katı bir şema uygular; uygulama ekibi her küçük değişiklik için haftalarca bekler. Veritabanı mükemmel, ürün yavaştır.
- Bir fonksiyon yüz kat hızlandırılır; ancak fonksiyon toplam sürenin yüzde birini almaktadır. Bütün için kazanç sıfıra yakındır.
- Her ekip kendi metriğini iyileştirir: satış ekibi çok satar, destek ekibi yük altında kalır.

Tuzağın nedeni görüş alanının darlığıdır. Kişi kendi parçasını görür, bütünü görmez; kendi parçasındaki iyileşme ölçülür, bütündeki zarar başkasının hanesine yazılır. Bir iyileştirmeden önce sorulacak soru şudur: bu parça bütünün darboğazı mı? Değilse iyileştirme bütüne bir şey katmaz.

Batık Maliyet

Batık maliyet (*sunk cost*), daha önce harcanmış ve hangi karar verilirse verilsin geri alınamayacak zaman ve kaynaktır. Rasyonel bir kararda batık maliyet hesaba katılmaz; karar yalnızca bugünden ileriye bakan maliyet ve kazançlara dayanır. Buna rağmen insanların harcanmış kaynağı gerekçe göstererek yanlış yolda devam etme eğilimi, davranışsal iktisatta batık maliyet yanılgısı (*sunk cost fallacy*) olarak adlandırılır (Arkes ve Blumer, 1985).

- Bir mimariye altı ay harcanmış, ardından yanlış olduğu anlaşılmıştır. “Altı ay boşa gitmesin” gerekçesiyle devam etmek, altı ayın üzerine altı ay daha eklemektir.
- Bakımı durmuş bir kütüphaneye bağımlılık kurulmuştur. “Çok emek verildi” duygusu değişimi engeller.

Doğru soru şudur: sıfırdan başlansaydı, bugün eldeki bilgiyle bu yol seçilir miydi? Cevap hayırsa, geçmişteki harcama yolu haklı çıkarmaz.

Zorluğun kaynağı bilgi eksikliği değil, duygudur. Vazgeçmek “yanıldım” demek anlamına gelir ve insanlar bundan kaçınır.

Sınırlı Rasyonellik

Sınırlı rasyonellik (*bounded rationality*), karar vericilerin tam bilgi, sınırsız zaman ve önyargısız bir zihinle değil; eksik bilgi, kısıtlı zaman ve bilişsel önyargılarla

karar verdiđini ifade eden kavramdır. Kavramı Simon (1955) geliřtirmiřtir. Bu bir kusur deđil, insan biliřinin alıřma biimidir ve karar vericinin kendisini de kapsar.

- Kiři kendi kodundaki hatayı grmez, bařkasınınkini hemen grr.
- Son yařanan kaza zihinde byk yer tutar; diđer riskler kmsenir.
- Yorgunluk risk algısını dřrr.

Bu nedenle karar srecine, kiřinin dikkatine gvenmeyen kontrol noktaları yerleřtirilir:

- **Kod incelemesi.** Yazan grmez, bařkası grr.
- **Kontrol listesi.** Hafızaya deđil, yazılı listeye gvenilir.
- **Otomatik test.** “Bence alıřıyor” yerine “makine alıřtıđını gsterdi.”
- **İki kiři onayı.** Tek kiřinin kr noktasının ikinci kiřiden gemesi gerekir.

İlkenin z řudur: “dikkatli olurum” bir kontrol yntemi deđildir. Dikkatli olunmayacađını varsayan bir sistem kurulur.

Anlama Yordamı

Değiştirmeden önce okumak

Kötü yazılmış bir kodla karşılaşan mühendiste düzeltme dürtüsü güçlüdür. Bu dürtünün kendisi bir sorun değildir; sorun sıralamadır. Kod değiştirilerek anlaşılmaya çalışıldığında, değiştiren kişi kendi varsayımlarını koda yazar ve sonra onları okur. Ortaya çıkan model, orada bulunan sistemin değil, okuyucunun kurmayı tercih ettiği sistemin modelidir. Bu yüzden ilk aşamada dokunmadan okumak, sonraki iyileştirmenin doğru zemine oturmasını sağlar.

Bunun tersini savunan bir gerekçe de vardır ve ihmal edilmemelidir: bazı kişiler için manipülasyon anlamının aracıdır. Bu durumda meşru bir orta yol, atılacak bir dalda (*throwaway branch*) küçük ve geri alınabilir dönüşümler yaparak okumak ve sonuçta değişikliği saklamamaktır. Böylece hem anlama biçimi korunur hem de üretim kodu etkilenmez.

Analiz kartı

Bir fonksiyonu anlamaya çalışırken sabit başlıklardan oluşan bir kart doldurmak, dağınık bilgiyi düzenli hâle getirir. Başlıklar şunlardır:

- Amaç: fonksiyonun ne yapıyor görüldüğü. Bu bir hipotezdir, kesinlik iddiası taşımaz.
- Girdiler ve çıktı.
- Yan etkiler: fonksiyonun kendi dönüş değeri dışında dış dünyada değiştirdiği her şey. Kayıt yazma, bildirim gönderme, paylaşılan bir yapıyı güncelleme.
- İş kuralları: hangi koşulda hangi kaydın elendiği, hangi eşğin neyi tetiklediği.
- Varsayımlar: kodun doğru kabul ettiği ama kontrol etmediği şeyler. Bir alanın her zaman dolu olduğu, bir değerin sayısal olduğu gibi.
- Anlaşılmayan noktalar: cevabı olmayan sorular. Bunların açıkça yazılması, sonradan kime hangi sorunun sorulacağını belirler.

Kartın işlevi cevap üretmek değil, bilinen ile bilinmeyeni ayırmaktır. Anlama sürecinde asıl yorucu olan, ikisinin karışmış hâlde zihinde tutulmasıdır.

Zihinsel yürütme ve sınırı

Kart dolduktan sonra, küçük ve elle uydurulmuş bir girdi kümesi üzerinde fonksiyonun kafada adım adım çalıştırılması yararlıdır. Örneğin üç elemanlı bir liste düşünülür: biri işlenir, biri **active** alanı yanlış olduğu için elenir, biri ilişkili kayıta **deleted** işaretli olduğu için elenir. Yalnızca bir kayıt işlendiğinde toplamın eşği aşmadığı, dolayısıyla bildirim çağrısının tetiklenmediği görülür. Bu kadar küçük bir yürütme bile dallanmanın iskeletini ortaya çıkarır.

Ancak zihinsel yürütmenin sınırı vardır ve bu sınır ciddidir. Kafada yürütme, okuyucunun kendi hipotezini doğrulamaya eğilimlidir; atlanan bir dal, fark edilmeyen bir kenar durum ya da yanlış varsayılan bir tip, yürütme sırasında görünmez. Gerçek sistemlerde daha güvenilir kanıt üreten araçlar şunlardır:

- Sürüm geçmişi. `git log` ve `git blame`, bir satırın hangi değişiklikle, hangi gerekçeyle ve hangi hata kaydına bağlı olarak eklendiğini gösterir. “Bu neden böyle yapılmış” sorusunun cevabı çoğu zaman kodun içinde değil, kodun tarihindedir.
- Testler. Varsa, testler kodun yazarının hangi davranışı kasıtlı saydığını gösterir; yani teorinin yazıya dökülmüş kısmıdır.
- Çalıştırma. Hata ayıklayıcıyla adım adım ilerlemek ya da geçici kayıt satırları eklemek, kafada yürütmenin ürettiği hipotezleri doğrudan sınar.

Zihinsel model bu araçlardan gelen kanıtla düzeltilir. Yalnızca kafada kurulan model, sınanmamış bir hipotezdir.

Uydurma örneğin yanıltıcılığı

Anlama yöntemi öğretilirken kullanılan örnek kodun, öğretmek amacıyla üretilmiş olması bir sorun yaratır. Kasıtlı olarak çirkinleştirilmiş bir fonksiyon, yine de tek bir zihinden çıkmıştır ve tutarlı bir mantığa sahiptir; parçaları temiz ayrışır. Gerçek eski kod böyle davranmaz: birden fazla kişinin, birbirinden habersiz ve farklı zamanlarda aldığı kararların üst üste binmesiyle oluşur, dolayısıyla tutarlı tek bir teorisi yoktur. Yöntemin gerçek zorluğu bu noktada başlar ve uydurma örnekle sınamamaz.

Kuraldan Bağlama Geçiş

Kuralların basitleştirilmiş olması

Yeni başlayanlara verilen kuralların bağlamsız olması bir eksiklik değil, bir tasarım tercihidir. Dreyfus ve Dreyfus (1986), beceri edinimini beş aşamalı bir model olarak tanımlamış ve acemi aşamasında kişinin bağlamdan bağımsız kurallar izlediğini, uzmanlık aşamasında ise durumu bütün olarak kavrayıp kurala başvurmadığını belirtmişlerdir. Kural, henüz durumu okuyamayan kişiye durumu okuma yükü bindirmeden güvenli davranmayı sağlar.

Bu yüzden kuralın sonradan yetersiz kalması, kuralın en baştan yanlış olduğu anlamına gelmez. Kural, belirli bir yeterlilik düzeyi için doğrudur.

Ödünleşim

Sahada kuralın yerini alan şey ödünleşimdir. Klasik mühendislik örnekleri şunlardır:

- Yapısal tasarımda “ne kadar sağlam o kadar iyi” kuralının yerini “gerektiği kadar sağlam” alır. Fazla sağlamlık ağırlık, malzeme maliyeti ve üretim zorluğu olarak geri döner; bu kalemler de bir sistemin başka yerlerinde performans kaybı yaratır.
- Yüksek hızlı rulmanlarda yağlama miktarının artırılması, belirli bir noktadan sonra viskoz sürtünmeyi ve dolayısıyla çalışma sıcaklığını yükselterek yağın bozulmasını hızlandırır ve rulman ömrünü kısaltır. “Daha çok yağ daha iyi yağlama” sezgisi burada tersine döner.

İkinci örnek, ödünleşimin en öğretici biçimidir: iki iyi şey arasında denge kurulmuyor, aynı büyüklüğün artırılması belirli bir eşikten sonra kendi amacını baltıyor. Bu tür tersine dönmeler, kuralın hangi aralıkta tanımlı olduğunun hiç öğretilmemiş olmasından kaynaklanır.

Uçları bilmek

Denge, iki ucun ortalaması değildir. Bilinçli bir denge kurabilmek için her iki uçta ne olduğunun bilinmesi gerekir: hangi noktadan sonra sistemin verimsizleştiği, hangi noktadan sonra güvenli olmaktan çıktığı. Uçları bilmeyen kişinin seçtiği orta nokta, bir tercih değil bir kaçınmadır.

Bu ilkenin iki sınırı vardır. Birincisi, uçlar çoğunlukla deneyimlenerek öğrenilmez; uçta ne olduğu okunarak ya da başkasının deneyiminden aktararak öğrenilir, çünkü bazı uçların bedeli denemeye elverişli değildir. İkincisi, her problemde orta nokta doğru cevap değildir. Bazı büyüklükler için doğru değer tam uçtadır: parolaların düz metin saklanma oranı, veri kaybına yol açan hataların kabul edilen sayısı, insan hayatını doğrudan tehlikeye atan durumların tolerans payı. Bu alanlarda “her şey ödünleşimdir” tavrı, bir olgunluk işareti değil, sert kısıtın tanınmamasıdır.

Kanaat Değişiminin Doğrulanması

Adlandırma problemi

Yerleşik bir kanaatin çöküp yerine yenisinin kurulması, birden fazla alanda ayrı ayrı adlandırılmıştır. Kuhn (1962), bilim tarihinde egemen çerçevenin birikmiş anomaliler karşısında yerini yeni bir çerçeveye bırakmasını paradigma değişimi olarak tanımlamıştır. Mezirow (1991), yetişkin eğitiminde bireyin mevcut anlam şemalarıyla açıklanamayan bir deneyim yaşamasını ve bunun ardından varsayımlarını gözden geçirmesini dönüştürücü öğrenme (*transformative learning*) olarak kavramsallaştırmış, sürecin başlangıcındaki sarsıcı deneyime yön kaybettiren ikilem (*disorienting dilemma*) adını vermiştir.

Bu kavramların ölçeği farklıdır ve bu fark önemlidir. Kuhn bilimsel bir topluluğun ortak çerçevesini, Mezirow bireyin dünya görüşü ve kimlik düzeyindeki dönüşümünü tarif eder. Mezirow'un kuramı ayrıca ampirik doğrulanabilirliği bakımından eleştirilmiştir. Bir mühendislik pratiği hakkında fikir değiştirmek, bu kavramların ölçeğinin altında kalır; olguya olduğundan ağır bir etiket takmak, olgunun ne olduğunu açıklamaz. Türkçede en yakın gündelik karşılık “ezberin bozulması” ifadesidir, ancak bu ifade yaygın kullanımda daha çok başkalarının beklentisinin boşa çıkarılmasını anlatır; kişinin kendi içselleştirdiği kuralın çökmesi için birebir oturmaz.

Hissin kanıt değeri taşınamaması

Kanaat değişimine eşlik eden yön kaybı hissi, öğrenmenin derinleştiğinin göstergesi olarak sunulmaya elverişlidir. Bu sunum yanıltıcıdır. Aynı his, üç farklı durumda da aynı biçimde ortaya çıkar:

- Kişi gerçekten kuralın sınırlarını görmüş ve daha geniş bir çerçeveye geçmiştir.
- Kişi kuralın kötü uygulanmış bir örneğiyle karşılaşmış ve kuralın kendisini geçersiz saymıştır.
- Kişi kuralı zaten yorucu buluyordur ve kuralı terk etmesini haklı çıkaran bir gözlemi fazla ağırlıklandırmıştır.

İçeriden bakıldığında üçü ayırt edilemez. Dolayısıyla “bu his beni büyütüyor” çıkarımı güvenli değildir. Ayırt edici sınama içsel değil dışsaldır: eski kuralın hangi işlevi gördüğü açıkça yazılır, sonra o işlevin yeni düzende hangi mekanizmayla karşılandığı gösterilir. İşlev karşılıksız kalıyorsa, olan şey bir derinleşme değil bir kaldırma işlemidir.

Örnek: inceleme pratiğinin terk edilmesi

Yazılımda yaygın bir örnek, her değişikliğin birleştirme öncesi harici bir onaydan geçmesi kuralının terk edilmesidir. Bu geçişin savunulabilir bir biçimi

vardır. Forsgren, Humble ve Kim (2018), DevOps araştırma programının bulgularını aktarırken, değişikliklerin harici bir onay kurulundan geçmesi zorunluluğunun teslimat performansıya olumlu bir ilişki göstermediğini, buna karşılık ekip içi ve sürekli inceleme biçimlerinin olumlu sonuç verdiğini belirtmişlerdir. Mekanizma açıktır: onay bir kapaıya dönüştüğünde değişiklikler birikir, birikince büyür, büyüyünce inceleyen kişi bağlamı kaybeder ve inceleme biçimsel bir onaya indirgenir. Bu durumda gecikme gerçek, güvence görünüştedir.

Savunulamaz biçimi ise inceleme kaldırıldığında onun gördüğü işlerin karşılıksız bırakılmasıdır. Bu işler en az dörttür: ikinci bir gözle hata yakalama, bilgi yayılımı ve tek kişiye bağımlılığın azaltılması, yeni katılan kişinin kalibrasyonu, ve denetim izi. Her biri başka bir mekanizmaya devredilebilir; eşli ya da topluca programlama, birleştirme sonrası inceleme, güçlü otomatik test ve sürekli entegrasyon, özellik bayrakları, değişikliklerin küçük tutulması. Devredilmediğinde kalan şey bir yöntem değil bir boşluktur.

Terk etmenin açıkça yanlış olduğu bağlamlar da vardır: güvenilmeyen dış katkıcıların bulunduğu projeler, denetim izinin kendisinin zorunlu olduğu düzenlenmiş alanlar, otomatik test kapsamının zayıf olduğu sistemler, kıdem dağılımının düşük olduğu ekipler ve zaman dilimi farkı nedeniyle eşzamanlı çalışmanın mümkün olmadığı dağıtık ekipler. Son iki koşul özellikle önemlidir, çünkü inceleme yerine önerilen ikamelerin çoğu eşzamanlılık gerektirir; eşzamanlılık yoksa ikame de yoktur.

Bu örnek, önceki bölümdeki sınamanın nasıl uygulandığını gösterir. Savunulabilir önerme “inceleme yapılmamalıdır” değil, “bu bağlamda bloklayıcı ve eşzamansız onay ölçülebilir zarar veriyor, yerine şu mekanizmalar konuyor” biçimindedir. Birinci önerme tartışlamaz, çünkü bağlam belirtmez; ikincisi tartışılabilir, çünkü yanlışlanabilir bir iddia içerir.

Sonuç

Kural düzeyinden bağlam düzeyine geçiş, bilgi eklenmesi değil, mevcut bilginin tanım aralığının keşfedilmesidir. Bu keşfin ham maddesi gerçek sistemlerdir; dolayısıyla yabancı ve kötü yazılmış kodu okuma pratiği, ayrı bir zahmet değil, geçişin kendisini besleyen işin ta kendisidir. Anlama yordamının değeri de burada belirginleşir: yordam kodu güzelleştirmeye değil, sistemin hangi kısıtlar altında bu hâle geldiğini görmeye hizmet eder.

Bu çerçevenin üç sınırı vardır. Birincisi, zihinsel yürütme tek başına zayıf bir kanıt aracıdır; sürüm geçmiş, testler ve çalıştırma olmadan kurulan model, okuyucunun beklentisinin kodun üzerine yansıtılmış hâli olabilir. İkincisi, ödünleşim çerçevesi her alana genişletilemez; sert kısıtların bulunduğu yerlerde denge arayışı hatanın kendisidir. Üçüncüsü, kanaat değişimine eşlik eden his, değişimin doğruluğu hakkında bilgi taşımaz; doğrulama ancak terk edilen kuralın işlevlerinin tek tek izlenmesiyle yapılır.

Açık kalan sorular şunlardır. Bir mühendisin kendi kanaat revizyonunu, kendi içinden bakarak sınamasının pratik bir yordamı kurulabilir mi; yoksa sınama zorunlu olarak dışsal mı kalır. Kuralların bağlamsız öğretilmesi acemi aşaması için gerekliyse, kuralın tanım aralığı hangi noktada ve hangi biçimde aktarılmalıdır. Ve terk edilen bir kuralın işlevlerinin karşılıksız kalıp kalmadığı, sonuçlar ortaya çıkmadan önce nasıl ölçülür.

Not boyunca kurulan ayrımın kendisi de aynı sınamaya tabidir. “Kural yoktur, bağlam vardır” önermesi, kuralı hiç içselleştirmemiş bir kişi tarafından benimsendiğinde, bağlamı okuma yetisi değil, tercih edileni seçme serbestisi üretir. Bu yüzden çerçevenin aktarımında sıra, içeriğin kendisi kadar belirleyicidir.

Kaynaklar

- Brooks, R. (1983). Towards a theory of the comprehension of computer programs. *International Journal of Man-Machine Studies*, 18(6).
- Craik, K. (1943). *The Nature of Explanation*. Cambridge University Press.
- Dreyfus, H. L. & Dreyfus, S. E. (1986). *Mind Over Machine*. Free Press.
- Forsgren, N., Humble, J. & Kim, G. (2018). *Accelerate: The Science of Lean Software and DevOps*. IT Revolution Press.
- Johnson-Laird, P. N. (1983). *Mental Models*. Harvard University Press.
- Kuhn, T. S. (1962). *The Structure of Scientific Revolutions*. University of Chicago Press.
- Letovsky, S. (1987). Cognitive processes in program comprehension. *Journal of Systems and Software*, 7(4).
- Mezirow, J. (1991). *Transformative Dimensions of Adult Learning*. Jossey-Bass.

- Naur, P. (1985). Programming as Theory Building. *Microprocessing and Microprogramming*, 15(5).
- Norman, D. A. (1983). Some Observations on Mental Models. In D. Gentner & A. L. Stevens (Eds.), *Mental Models*. Erlbaum.
- von Mayrhauser, A. & Vans, A. M. (1995). Program Comprehension During Software Maintenance and Evolution. *IEEE Computer*, 28(8).